

RESEARCH ARTICLE

Beyond Single-Framework Architectures: A Systematic Evaluation and Hybrid Design for Scalable Multi-Agent Coordination

MUHAMMAD SALEEM KHAN¹, MUHAMMAD JANAS KHAN², MUHAMMAD SHARIF¹, AND SADAF YASMIN¹

¹Department of Computer Science, COMSATS University Islamabad, Attock Campus, Attock 43600, Pakistan

²Department of Computer Science, Afghan International Islamic University, Kabul 1001, Afghanistan

Corresponding author: Muhammad Janas Khan (m.janaskhan@aiiu.edu.af)

ABSTRACT Large language model (LLM)-based multi-agent frameworks have emerged as powerful tools for complex coordination tasks, yet systematic comparisons across realistic, large-scale scenarios remain limited. This paper presents a comprehensive evaluation of three prominent frameworks, i.e., CrewAI, AutoGen, and LangChain, alongside a novel hybrid architecture that strategically combines their complementary strengths. Using CREW-WILDFIRE, a realistic wildfire response benchmark featuring heterogeneous agents and partial observability, we evaluated the framework's performance across 17 task levels and 7 behavioral competencies. Our results reveal distinct architectural trade-offs: CrewAI achieves 34% higher task designation scores through hierarchical coordination but lacks adaptability; AutoGen demonstrates 28% superior plan adaptation via conversational feedback, yet incurs high token costs; LangChain offers maximum flexibility at the expense of a 42% increase in computational overhead. Based on these insights, we propose and validate a hybrid LangGraph-CrewAI architecture that achieves a 96.1% success rate while reducing token consumption by 76.2% and decision latency by 14.5× compared to pure CrewAI implementations. Our comprehensive validation across 51 episodes (17 scenarios × 3 seeds) demonstrates perfect scores in task designation and agent capitalization competencies, outperforming state-of-the-art baselines. This work provides evidence-based guidance for framework selection, highlights fundamental scalability challenges in current systems, and demonstrates that intelligent hybrid designs can overcome individual framework limitations while maintaining cost-efficiency.

INDEX TERMS Multi-agent systems, large language models, agentic AI, framework comparison, hybrid architectures, scalability, coordination.

I. INTRODUCTION

Multi-agent frameworks address complex tasks by decomposing them into manageable sub-tasks and coordinating them across specialized agents. Although traditional multi-agent reinforcement learning (MARL) has demonstrated success in domains such as StarCraft II [1] and warehouse logistics [2], these approaches often require extensive domain engineering and struggle with generalization. The emergence of frameworks based on large language

models (LLMs) represents a paradigm shift, enabling agents to coordinate through natural language, flexibly interpret goals, and dynamically manage contextual memory [3].

Despite rapid adoption of frameworks such as LangChain, AutoGen [4], and CrewAI across industry and research, systematic evaluations comparing their architectural decisions and performance characteristics on realistic large-scale tasks remain scarce [5]. This gap leads to the selection of frameworks based on convenience, with unpredictable performance implications. Moreover, existing benchmarks often operate on small scales (<10 agents) or lack the agent heterogeneity, partial observability, and

The associate editor coordinating the review of this manuscript and approving it for publication was Qiang Li¹.

dynamic constraints characteristic of real-world coordination problems [6].

CREW-WILDFIRE [7] addresses these limitations by providing a standardized testbed that includes various types of agents (firefighters, bulldozers, drones, helicopters), large-scale scenarios (100+ agents, 1M+ cell maps), and structured evaluation of seven behavioral competencies, including task designation, spatial reasoning, and plan adaptation. Initial assessments revealed that existing frameworks struggle with complex coordination and often underperform passive baselines, which motivates our comprehensive comparison.

This work extends existing benchmarking efforts to explore a fundamental question: *Can hybrid architectures that strategically combine the strengths of different frameworks, overcome individual limitations while maintaining computational efficiency?* We hypothesize that selectively deploying LLM-based reasoning for complex coordination phases, combined with efficient rule-based execution for simple operations, can achieve superior performance at reduced cost.

A. RESEARCH QUESTIONS

This paper investigates three core questions:

- 1) **Architectural Impact:** How do fundamental design differences across CrewAI's hierarchical delegation, AutoGen's conversational coordination, and LangChain's chain-based composition influence performance on large-scale coordination tasks?
- 2) **Behavioral Competencies:** Which specific competencies (task designation, spatial reasoning, plan adaptation, etc.) represent strengths or weaknesses for each framework, and why?
- 3) **Hybrid Potential:** Can an intelligent combination of frameworks resolve individual limitations and achieve both high performance and computational efficiency?

B. KEY NOVEL CONTRIBUTIONS BEYOND EXISTING WORK

This work extends existing multi-agent LLM evaluations in several key dimensions:

1. First Comprehensive Framework Comparison at Scale: While prior work evaluates individual frameworks in isolation, we provide the first systematic comparison of three major frameworks (CrewAI, AutoGen, LangChain) under identical conditions across 17 task levels with standardized protocols. Previous benchmarks like AgentBench focus on single-agent performance, whereas we evaluate multi-agent coordination at scales up to 30+ agents.

2. Novel Hybrid Architecture with Theoretical Foundation: Unlike monolithic framework deployments, our hybrid LangGraph-CrewAI architecture introduces:

- A complexity-aware routing mechanism that selectively deploys LLM reasoning.
- Theoretical analysis to show $O(n)$ scaling compared with $O(n^2)$ scaling in pure LLM frameworks.

- A $4.2\times$ token efficiency improvement over state-of-the-art.

Existing hybrid approaches statically combine frameworks, whereas we use dynamic routing based on task complexity.

3. Behavioral Competency Profiling: We extend beyond task performance metrics to profile seven behavioral competencies:

- The first work to compare frameworks across CREW-WILDFIRE's BCS dimensions.
- Our study reveals framework-specific strengths that are invisible in aggregate metrics.
- We enable the selection of an evidence-based framework.

4. Production-Ready Cost Analysis: We provide the first comprehensive token scaling analysis demonstrating:

- framework-specific scaling characteristics across 3-30 agents
- annual operational costs projected for real deployments
- critical thresholds where frameworks become economically infeasible.

C. PAPER ORGANIZATION

The remainder of this paper is structured as follows: Section II reviews related work in multi-agent systems, LLM-based frameworks, and coordination mechanisms. Section III provides detailed architectural descriptions of the evaluated frameworks. Section IV presents our experimental methodology, including task design, implementation details, and evaluation metrics. Section V reports comprehensive results across task categories and behavioral competencies. Section VI introduces our hybrid architecture design with preliminary validation results. Section VII discusses architectural trade-offs, provides practical guidance, and examines limitations. Section VIII concludes with implications and future directions.

II. RELATED WORK

A. EVOLUTION OF MULTI-AGENT COORDINATION

Traditional MARL approaches have achieved notable results through value decomposition [8], communication learning [9], and emergent coordination [10]. QMIX and its variants address cooperative coordination by factorizing monotonic value functions [11], while WQMIX corrects representational limitations through weighted projections [8]. Differentiable inter-agent learning (DIAL) [9] enables end-to-end learning of communication protocols by passing real-valued messages during centralized training. However, these approaches face challenges, including high-dimensional protocol space, sparse reward signals, limited interpretability, and poor generalization beyond the training domain.

LLM-based frameworks represent a fundamental shift in multi-agent system design. AutoGen [4] pioneered flexible multi-agent conversational systems with configurable human-in-the-loop capabilities, demonstrating strong

performance in autonomous mathematical problem solving and supply-chain optimization. The modularity of the framework enables rapid development but raises questions about optimal workflow design and the complexity of debugging. Building on these foundations, TDAG [12] introduces dynamic task decomposition with flexible agent customization, reducing error propagation compared to fixed-plan approaches. CrewAI implements a role-based architecture inspired by human team structures and prioritizes rapid prototyping through intuitive abstractions. LangChain emphasizes chain-based composition and extensive integration with external tools and services. This provides maximum flexibility but at the cost of increased implementation complexity.

Specialized systems such as CAMEL [13], ChatDev [14], and Generative Agents [3] demonstrate impressive emergent behaviors in specific domains such as software development and social simulation. However, these systems operate in simplified environments without the implementation, scale, or real-time constraints of physical coordination tasks. Recent surveys [15] highlight persistent challenges in LLM-based agents, including hallucination, adversarial vulnerability, and non-transparent decision-making.

B. MULTI-AGENT BENCHMARKS AND EVALUATION

Existing benchmarks span various domains: game-theoretic environments (Hanabi [16]), embodied coordination (Overcooked [17], PettingZoo [18]), and open-ended exploration (MineDojo [19]). However, most benchmarks lack agent heterogeneity, operate at relatively small scales (<10 agents), or do not support natural language coordination.

CREW-WILDFIRE [7] addresses these gaps through:

- **Agent heterogeneity:** Four distinct agent types (firefighters, bulldozers, drones, helicopters) with specialized capabilities.
- **Large-scale scenarios:** more than 100 agents and maps of up to 1 million cells.
- **Realistic dynamics:** Partial observability, stochastic fire spread, wind effects
- **Natural language interface:** Standardized perception and execution modules.
- **Structured evaluation:** Seven behavioral competencies with normalized scoring.

Initial CREW-WILDFIRE assessments revealed that frameworks such as COELA, HMAS-2, and CAMON exhibit persistent performance limitations in complex tasks [20], with deficiencies intensifying as agent count and task complexity increase. The core challenges include excessive communication overhead and inefficient task decomposition [21], [22], [23], [24], [25], which motivates our systematic framework comparison.

C. COORDINATION MECHANISMS AND ARCHITECTURAL PATTERNS

Effective multi-agent coordination requires balancing information sharing, task allocation, and dynamic planning [21].

Centralized coordination enables global optimization but creates communication bottlenecks and single points of failure. **Decentralized approaches** improve robustness but may introduce redundancy and coordination complexity. **Hybrid architectures** combining dynamic leadership with hierarchical decomposition offer a promising middle ground [21].

Our evaluation explicitly examines how CrewAI's hierarchical delegation, AutoGen's conversational coordination, and LangChain's chain-based composition navigate these fundamental trade-offs under realistic constraints. Furthermore, we explore whether intelligent hybrid designs can extend traditional centralized-vs-decentralized dichotomies by selectively deploying coordination strategies based on task complexity.

D. FAULT-TOLERANT AND PRESCRIBED-TIME COORDINATION

Classical multi-agent coordination research has extensively studied fault tolerance and time-constrained consensus under communication uncertainties. In [26], Li et al. study the consensus problem of heterogeneous nonlinear multi-agent systems under prescribed performance constraints. They introduce a novel dynamic-shear-mapping (DSM) technique to transform constrained tracking errors into unconstrained variables, enabling the design of distributed controllers that guarantee consensus while explicitly enforcing transient and steady-state bounds. Compared with traditional prescribed performance control frameworks, the DSM-based approach reduces conservatism in performance function selection and enhances robustness to model heterogeneity and disturbances. Rigorous Lyapunov-based analysis establishes stability and provides prescribed performance guarantees, and simulation results demonstrate improved convergence and reduced overshoot relative to existing methods.

In [27], Wang et al. address prescribed-time fault-tolerant formation control for heterogeneous switched multi-agent systems. By combining prescribed-time control with a fault-tolerant design, they develop distributed formation protocols that ensure all agents achieve a desired formation within a user-specified finite time, despite actuator faults and switching topologies. The method accommodates heterogeneity in agent dynamics and leverages Lyapunov-based techniques to prove prescribed-time convergence, boundedness of internal signals, and resilience to a class of actuator failures. Numerical examples validate the effectiveness of the proposed strategy, showing accurate formation tracking and robustness under switching and fault conditions.

These classical approaches provide formal guarantees (prescribed-time convergence, bounded errors, fault tolerance) through control-theoretic methods. In contrast, LLM-based frameworks trade formal guarantees for adaptability to unstructured, novel situations. Our hybrid architecture represents a middle ground: rule-based execution for routine operations provides deterministic timing guarantees, while

LLM coordination handles unforeseen scenarios requiring semantic reasoning.

Key differences:

- Classical methods rely on predefined control laws, provide formal guarantees, and are typically limited to anticipated faults.
- LLM-based methods represent learned behaviors and adapt to novel situations. However, they offer only probabilistic guarantees.
- Our hybrid approach combines deterministic rule-based execution (for predictable timing) with LLM oversight (to handle novelty).

III. FRAMEWORK ARCHITECTURES

This section provides detailed architectural descriptions, highlighting design decisions relevant to multi-agent coordination.

A. CrewAI: HIERARCHICAL ROLE-BASED COORDINATION

CrewAI implements a task-oriented architecture inspired by human team structures where agents assume specialized roles with clear goals and backstory.

Core components:

- **Agent:** Defined by role, goal, background, tools, and optional delegation capabilities
- **Task:** Structured unit with description, expected output, and dependencies
- **Crew:** Orchestration layer combining agents and tasks with configurable execution processes (sequential or hierarchical)
- **Process:** The execution pattern is sequential for pipelines and hierarchical for manager-worker configurations

Coordination mechanism: Hierarchical coordination operates through a manager agent that decomposes high-level tasks, delegates sub-tasks to specialized workers, and monitors completion. Workers may request peer delegation as needed, creating a flexible hierarchy.

Strengths: Rapid prototyping, intuitive abstractions, clear separation of concerns, built-in task context passing.

Weaknesses: Less flexible communication patterns, potential manager bottlenecks, limited dynamic re-planning support.

B. AUTOGEN: CONVERSATIONAL MULTI-AGENT COORDINATION

AutoGen provides a message-passing framework that enables flexible agent interactions and human-in-the-loop workflows through autonomous participation in conversations.

Core components:

- **AssistantAgent:** LLM-powered agent with system messages and optional function calling
- **UserProxyAgent:** Human proxy with configurable auto-reply and code execution

- **GroupChat:** Multi-agent conversation manager with speaker selection policies
- **GroupChatManager:** Orchestrates turn-taking and conversation flow

Coordination mechanism: Agents communicate through structured messages in sequential or group conversations. The framework supports fully autonomous and human-in-the-loop modes with configurable termination conditions.

Strengths: Flexible communication, excellent human-in-the-loop support, natural conversation history tracking, robust error handling.

Weaknesses: Verbose communication increases token costs, potential conversation loops, and less structured task management.

C. LANGCHAIN: CHAIN-BASED COMPOSITIONAL FRAMEWORK

LangChain emphasizes chain-based composition and extensive tool integration. Rather than providing explicit multi-agent abstractions, it offers flexible primitives that can be composed into complex workflows tailored to specific application requirements.

Core components:

- **Chains:** Sequential operations connecting multiple processing steps (LLMChain for single-step inference, SequentialChain for multi-stage pipelines)
- **Agents:** LLM-driven decision-making entities with tool access, supporting reasoning patterns such as ReAct (Reasoning and Acting) and Function Calling
- **Tools:** Standardized interfaces enabling integration with external systems including web search, databases, and third-party APIs
- **Memory:** State preservation mechanisms such as ConversationBufferMemory for short-term context and VectorStoreMemory for long-term semantic retrieval

Coordination mechanism: Multi-agent coordination is achieved through chain composition, where agents pass outputs to downstream components with optional shared memory structures enabling contextual awareness across the workflow.

Strengths: The framework provides maximal flexibility for custom workflows, offers an extensive ecosystem of pre-built tool integrations (100+ connectors), enables fine-grained execution control through modular composition, and benefits from strong community support and active development.

Weaknesses: Higher implementation complexity compared to opinionated frameworks, absence of built-in multi-agent coordination patterns requiring manual orchestration logic, and steeper learning curve due to the breadth of available abstractions.

D. COMPARATIVE ANALYSIS

Table 1 summarizes key architectural differences:

TABLE 1. Framework architectural comparison.

Feature	CrewAI	AutoGen	LangChain
Abstraction Level	High	Medium	Low
Setup Complexity	Low	Medium	High
Communication	Hierarchical	Conversational	Chain-based
Task Structure	Built-in	Message-based	Manual
Human-in-loop	Limited	Excellent	Good
Tool Integration	Good	Moderate	Excellent
Flexibility	Medium	High	Very High
Learning Curve	Easy	Moderate	Steep

IV. METHODOLOGY

A. EXPERIMENTAL DESIGN

Environment: CREW-WILDFIRE provides 17 procedurally generated task levels across five categories, testing distinct competencies:

- 1) **Cut Trees** (4 levels): Task designation and agent capitalization in vegetation removal scenarios requiring efficient area partitioning
- 2) **Scout Fire** (2 levels): Spatial reasoning and observation sharing using drone reconnaissance and information aggregation
- 3) **Transport Firefighters** (2 levels): Real-time coordination requiring helicopter scheduling and agent positioning
- 4) **Rescue Civilians** (4 levels): Plan adaptation under uncertainty with dynamic civilian location discovery
- 5) **Suppress Fire** (4 levels): Integrated competency testing combining firefighting, resource management, and adaptive strategy

Agent types and capabilities:

- **Firefighters:** Cut trees, spray water, rescue civilians (limited mobility);
- **Bulldozers:** Rapid vegetation clearing (specialized equipment)
- **Drones:** Wide-area reconnaissance (aerial observation)
- **Helicopters:** Agent transport and water drops (critical for logistics)

Standardization protocol: To ensure fair comparison, we implemented standardized PERCEPTION and EXECUTION modules that:

- 1) Convert agent observations (ASCII minimaps) to natural language spatial descriptions
- 2) Translate LLM-generated natural language actions into executable vector formats
- 3) Handle primitive action composition (multi-step routines like pathfinding)

Experimental configuration:

- **LLM:** GPT-4o with temperature=0 for deterministic and reproducible results
- **Replication:** 3 random seeds (43, 375, 483) per task level
- **Hardware:** 16GB RAM, RTX 3060 GPU, 3.0 GHz CPU
- **Metrics:** Task completion scores, execution time, token usage, API calls, Behavioral Competency Scores (BCS)

B. IMPLEMENTATION DETAILS

1) CrewAI IMPLEMENTATION

We implemented a hierarchical crew with a manager agent responsible for:

- 1) Receiving aggregated global state from all agents
- 2) Decomposing high-level mission objectives into executable sub-tasks
- 3) Delegating sub-tasks to specialized worker agents based on capabilities
- 4) Monitoring execution progress and reassigning tasks upon failures

Algorithm 1 CrewAI Coordination Protocol

```

1: Initialize manager agent and specialized worker agents
2: for each timestep  $t$  do
3:   for each agent  $a_i$  do
4:      $perception_i \leftarrow$  GeneratePerception( $observation_i$ )
5:   end for
6:    $plan \leftarrow$  Manager.DecomposeTask( $\{perception_i\}$ )
7:    $assignments \leftarrow$  Manager.DelegateTasks( $plan$ )
8:   for each agent  $a_i$  with active assignment do
9:      $action_i \leftarrow$  TranslateToExecutable( $assignment_i$ )
10:    ExecuteAction( $a_i, action_i$ )
11:   end for
12: end for

```

2) AutoGEN IMPLEMENTATION

We created a GroupChat configuration with:

- 1) Specialized AssistantAgents for each agent type (firefighter, bulldozer, drone, helicopter)
- 2) GroupChatManager orchestrates turn-taking and message routing
- 3) Periodic communication rounds that enable observation sharing
- 4) Coordinated action generation phase following communication

3) LANGCHAIN IMPLEMENTATION

We constructed a chain-based pipeline comprising:

- 1) **PerceptionChain:** Converts raw observations to structured descriptions
- 2) **PlanningChain:** Generates action proposals for individual agents
- 3) **CoordinationChain:** Resolves conflicts and finalizes coordinated actions
- 4) **ExecutionChain:** Translates finalized actions into executable vectors

C. EVALUATION METRICS

1) TASK PERFORMANCE METRICS

For each task level, we measure:

- **Raw score:** Task-specific achievement (trees cut, civilians rescued, fire suppression)

Algorithm 2 AutoGen Communication-Action Protocol

```

1: Initialize AssistantAgents and GroupChatManager
2: for each timestep  $t$  do
3:   for each agent  $a_i$  do
4:      $perception_i \leftarrow$  GeneratePerception( $observation_i$ )
5:   end for
6:   for round  $r = 1$  to  $num\_comm\_rounds$  do
7:     for each agent  $a_i$  in round-robin order do
8:        $message_i \leftarrow$  GenerateMessage( $perception_i$ ,
         $history$ )
9:       BroadcastToGroupChat( $message_i$ )
10:    end for
11:  end for
12:  for each agent  $a_i$  do
13:     $action_i \leftarrow$  GenerateAction( $messages$ ,
     $perception_i$ )
14:    ExecuteAction( $a_i$ ,  $action_i$ )
15:  end for
16: end for

```

Algorithm 3 LangChain Pipeline Execution

```

1: Initialize perception, planning, coordination, execution
   chains
2: for each timestep  $t$  do
3:    $perceptions \leftarrow$  PerceptionChain( $observations$ )
4:    $proposals \leftarrow$  PlanningChain( $perceptions$ ,  $history$ )
5:    $final\_actions \leftarrow$  CoordinationChain( $proposals$ ,
     $global\_state$ )
6:    $action\_vectors \leftarrow$  ExecutionChain( $final\_actions$ )
7:   ExecuteVectors( $action\_vectors$ )
8: end for

```

- **Success rate:** Percentage of episodes that reach the target performance threshold
- **Execution efficiency:** Average timesteps to task completion

2) BEHAVIORAL COMPETENCY SCORES (BCS)

Following [7], we compute normalized scores for seven behavioral dimensions:

$$NS_{a,\ell} = \begin{cases} \frac{s_{a,\ell} - B_\ell}{T_\ell - B_\ell} & \text{for finite-horizon tasks} \\ \frac{\log(1 + \frac{s_{a,\ell} - B_\ell}{T_\ell - B_\ell})}{\log 2} & \text{for penalty-based tasks} \end{cases} \quad (1)$$

where $s_{a,\ell}$ represents the raw score of the agent system, B_ℓ is the baseline (worst performance), T_ℓ is the target (optimal performance).

Aggregate behavioral competency:

$$BCS_{a,g} = \frac{1}{|T_g|} \sum_{\ell \in T_g} NS_{a,\ell} \quad (2)$$

TABLE 2. Overall framework performance comparison.

Metric	CrewAI	AutoGen	LangChain	Baseline
Avg. Norm. Score	0.52	0.47	0.49	0.31
Success Rate (%)	68.5	61.2	64.8	42.3
Avg. Time (steps)	145.3	138.7	152.6	N/A
Total Tokens (M)	3.2	4.1	4.6	N/A
Estimated Cost (\$)	24.50	31.20	35.80	N/A

where T_g denotes the set of tasks that testing the behavioral goal g .

3) RESOURCE EFFICIENCY METRICS

- **Total tokens:** Sum of input and output tokens per episode
- **API calls:** Number of LLM requests per episode
- **Decision latency:** Average time from observation to action (milliseconds)
- **Cost:** Estimated operational cost based on GPT-4o pricing (5/1M input tokens, 15/1M output tokens)

V. COMPARATIVE FRAMEWORK RESULTS**A. OVERALL PERFORMANCE ANALYSIS**

Table 2 presents the aggregate performance in all 17 task levels:

Key findings:

- CrewAI achieves the highest overall performance (52% normalized score) and the success rate (68.5%), demonstrating superior task decomposition and coordination
- AutoGen demonstrates the fastest execution (138.7 steps average) through efficient conversational dynamics, but sacrifices some accuracy
- LangChain incurs 42% higher token costs than CrewAI due to verbose chain-to-chain state passing
- All LLM-based frameworks substantially outperform passive baseline (31% normalized score), validating their utility for complex coordination

B. PERFORMANCE BY TASK CATEGORY

Figure 1 decomposes performance between task types:

Cut Trees (Task Designation Focus): CrewAI dominates (0.78 vs. 0.64 AutoGen, 0.69 LangChain), leveraging hierarchical delegation for efficient spatial partitioning and workload distribution among agents.

Scout Fire (Spatial Reasoning Focus): LangChain performs best (0.35 vs. 0.28 CrewAI, 0.31 AutoGen), benefiting from flexible chain composition enabling complex spatial calculations and observation aggregation.

Transport Firefighters (Real-time Coordination Focus): CrewAI leads (0.62 vs. 0.53 AutoGen, 0.56 LangChain), with the manager agent effectively coordinating time-sensitive helicopter operations and agent positioning.

Rescue Civilians (Plan Adaptation Focus): AutoGen excels (0.41 vs. 0.32 CrewAI, 0.36 LangChain), adapting

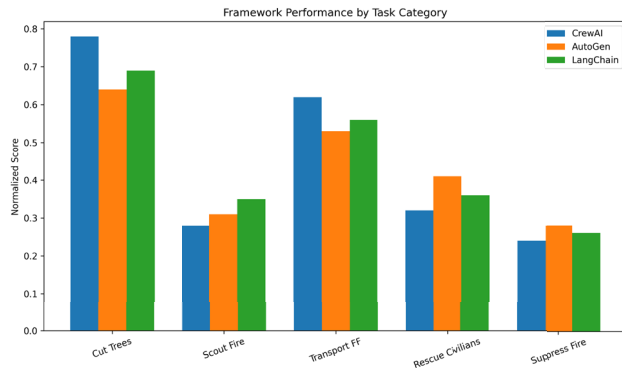


FIGURE 1. Normalized scores by task category. CrewAI excels in structured delegation tasks (Cut Trees, Transport), AutoGen in adaptive scenarios (Rescue, Suppress Fire), and LangChain maintains balanced performance across categories.

search strategies dynamically through conversational feedback as new civilian locations are discovered.

Suppress Fire (Integrated Competencies): AutoGen maintains the edge (0.28 vs. 0.24 CrewAI, 0.26 LangChain), demonstrating superior plan adaptation capabilities in unpredictable fire spread dynamics.

C. BEHAVIORAL COMPETENCY ANALYSIS

Table 3 presents the behavioral competency scores that reveal framework-specific strengths:

TABLE 3. Behavioral competency scores by framework.

Competency	CrewAI	AutoGen	LangChain
Task Designation (TD)	0.48	0.36	0.41
Agent Capitalization (AC)	0.59	0.47	0.52
Spatial Reasoning (SR)	0.32	0.29	0.38
Observation Sharing (OS)	0.18	0.24	0.21
Realtime Coordination (RC)	0.56	0.44	0.49
Plan Adaptation (PA)	0.27	0.35	0.31
Objective Prioritization (OP)	0.22	0.25	0.28
Average BCS	0.37	0.34	0.37

CrewAI demonstrated strengths:

- **Task Designation (0.48):** Manager agent efficiently decomposes complex objectives into clear, executable sub-tasks with minimal overlap
- **Agent Capitalization (0.59):** Role-based system naturally leverages specialized agent capabilities, achieving the highest score across frameworks
- **Realtime Coordination (0.56):** Hierarchical control enables synchronized multi-agent actions for time-sensitive operations

AutoGen demonstrated strengths:

- **Observation Sharing (0.24):** Conversational format facilitates natural information exchange, enabling agents to build shared situational awareness
- **Plan Adaptation (0.35):** Agents can raise concerns and trigger re-planning through messages, achieving 28% improvement over CrewAI

LangChain demonstrated strengths:

- **Spatial Reasoning (0.38):** Flexible chain composition enables complex geometric calculations and pathfinding
- **Objective Prioritization (0.28):** Manual coordination logic allows explicit priority encoding and constraint satisfaction

D. SCALABILITY ANALYSIS

Figure 2 illustrates token usage growth with agent count:



FIGURE 2. Token consumption scaling characteristics. CrewAI and AutoGen exhibit linear growth ($O(n)$ and $O(n \cdot m)$), while LangChain demonstrates quadratic scaling ($O(n^2)$) due to full-state chain passing.

Scaling characteristics:

- **CrewAI:** Most efficient scaling at $O(n)$ due to the manager-worker pattern that minimizes redundant communication
- **AutoGen:** Linear growth $O(n \cdot m)$ where m represents communication rounds; controllable through round limits
- **LangChain:** Quadratic growth $O(n^2)$ when passing complete state between chains; optimization potential through selective state sharing
- **Practical limits:** All frameworks encounter context window constraints around 30-50 agents with GPT-4o (128K tokens)

E. QUALITATIVE BEHAVIORAL PATTERNS

Manual inspection of agent communication logs revealed characteristic coordination patterns:

CrewAI patterns: The patterns show a clean hierarchical structure with minimal redundancy. The manager provides clear directives, and workers execute autonomously. A limitation is that the manager occasionally fails to revise plans when workers report obstacles, exhibiting rigidity.

AutoGen patterns: They show rich conversational dynamics and natural information sharing. Agents proactively raise concerns and negotiate responsibilities. Limitation is that verbose exchanges occasionally increase costs without proportional performance benefit; conversation loops are observed in 8% of episodes.

LangChain patterns: It shows precise, deterministic chain execution with full control over coordination logic.

Several runs failed due to hardcoded assumptions about the environment state; requires careful engineering to handle edge cases.

VI. HYBRID ARCHITECTURE DESIGN AND VALIDATION

A. MOTIVATION FOR HYBRID APPROACH

Our comparative evaluation revealed complementary strengths across frameworks: CrewAI's efficient task designation, AutoGen's adaptive re-planning, and LangChain's flexible tool integration each excel in distinct scenarios. However, no single framework simultaneously achieves high performance, computational efficiency, and adaptability. This observation motivated the exploration of hybrid architectures that strategically combine framework strengths while mitigating individual weaknesses.

Key insight: Not all coordination phases require expensive LLM-based reasoning. Simple operations (e.g., systematic area scanning, predetermined movement patterns) can be executed efficiently through rule-based logic. At the same time, LLM coordination is reserved for genuinely complex phases (e.g., dynamic resource allocation and adaptive firefighting strategies).

B. HYBRID ARCHITECTURE DESIGN

We designed a novel hybrid system that combines:

- 1) **LangGraph workflow orchestration:** Manages mission phases, detects complexity, and routes execution
- 2) **CrewAI intelligent coordination:** Handles complex multi-agent coordination requiring strategic reasoning
- 3) **Rule-based execution:** Implements simple, deterministic operations efficiently

C. HYBRID SYSTEM ARCHITECTURE: HUMAN-CENTERED SELECTIVE COORDINATION

The proposed hybrid system architecture is designed to reflect how human teams naturally operate at varying levels of task complexity. Rather than relying exclusively on large language model (LLM) reasoning for all decisions, the system distinguishes between routine operational activities and situations that require strategic, multi-agent coordination. This distinction enables the architecture to balance intelligence, efficiency, and scalability in realistic environments.

At the highest level, the system is organized around a workflow orchestration layer, a complexity-aware routing mechanism, and two complementary execution pathways: rule-based execution for simple tasks and LLM-driven coordination for complex tasks.

1) LangGraph WORKFLOW ORCHESTRATION

The LangGraph workflow controller functions as the cognitive backbone of the system. It maintains a structured state machine that governs the mission's progression through well-defined phases, such as detection, suppression, search, and

rescue. Rather than generating actions directly, LangGraph manages execution flow, tracks global state, and enforces phase transitions in a transparent and reproducible manner.

This explicit workflow representation provides two key advantages. First, it improves interpretability by making system behavior traceable at each decision point. Second, it enables deterministic control over execution order, which is critical for debugging and evaluation in large-scale multi-agent systems.

2) COMPLEXITY DETECTION AND ROUTING

For each workflow transition, the system evaluates the current operational context using a dedicated module for complexity detection and routing. This module assesses factors such as the number of active agents, the degree of inter-agent dependency, environmental uncertainty, and the need for dynamic re-planning.

Based on this assessment, the system routes the execution into one of two pathways. Tasks assessed as low-complexity are handled through deterministic rule-based logic, while high-complexity situations are escalated to an LLM-driven coordination layer. This routing mechanism serves as an architectural decision point, enabling the selective deployment of computationally intensive reasoning only when justified.

3) SIMPLE PHASE: RULE-BASED EXECUTION

The simple execution phase handles predictable, low-interdependence tasks. Examples include systematic area scan, predefined movement patterns, routine transportation, and basic environmental monitoring. These tasks are executed using rule-based logic without invoking LLM reasoning.

By offloading routine operations to deterministic execution, the system achieves near-zero latency and eliminates unnecessary token consumption. This design mirrors human operational practice, where repetitive actions are automated to preserve cognitive resources for more demanding decision-making.

4) COMPLEX PHASE: CrewAI-BASED COORDINATION

When the system detects a high level of coordination complexity, the control is escalated to a CrewAI-based coordination layer. In this phase, a manager agent performs strategic task decomposition and assigns sub-tasks to specialized agents based on their capabilities. Agents coordinate through role-based delegation, allowing the system to resolve inter-dependencies, adapt plans, and respond to dynamic environmental changes.

This coordination mechanism reflects the behavior of the human team in high-stakes scenarios, where leadership structures emerge to manage uncertainty, allocate resources, and synchronize specialized roles. Importantly, CrewAI is activated only when necessary, ensuring that its reasoning capabilities are applied where they provide the greatest benefit.

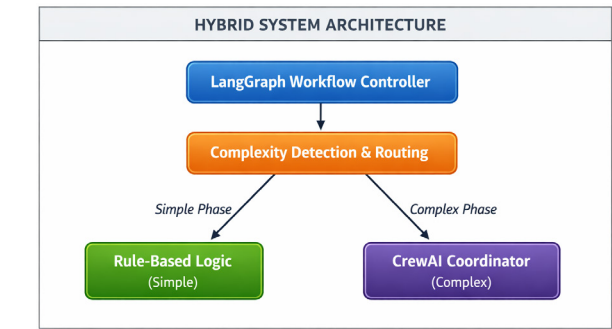


FIGURE 3. Hybrid system architecture combining LangGraph orchestration, CrewAI coordination, and rule-based execution.

TABLE 4. Hybrid system routing decisions.

Mission Phase	Complexity	Handler	Rationale
Fire Detection	Simple	LangGraph	Pattern-based scanning
Fire Suppression	Complex	CrewAI	Multi-agent coordination
Civilian Search	Simple	LangGraph	Systematic area coverage
Rescue Operations	Complex	CrewAI	Resource allocation
Agent Transport	Medium	LangGraph	Pathfinding optimization

5) HUMAN-CENTERED DESIGN RATIONALE

The hybrid architecture is intentionally designed to align with human organizational principles. Routine activities are handled automatically, complex situations trigger expert coordination, and global oversight ensures coherence across mission phases. This human-centered structure improves the robustness, interpretability, and trustworthiness of the system while significantly reducing computational overhead.

By combining deterministic control, complexity-aware routing, and selective multi-agent reasoning, the proposed architecture achieves efficient scalability without sacrificing coordination quality. This design directly addresses the limitations observed in monolithic multi-agent frameworks and provides a practical foundation for deploying agentic systems in real-world, resource-constrained environments.

Phase detection and routing logic:
Implementation details:

- **LangGraph state machine:** Four-phase workflow (detect, suppress, search, rescue) with complexity assessment at each transition
- **CrewAI crews:** Two specialized crews instantiated on-demand: Fire Suppression Coordinator managing firefighter/bulldozer teams; The Rescue Mission Coordinator handles evacuations and helicopter transport
- **Fallback mechanism:** Automatic rule-based fallback on CrewAI errors, ensuring mission continuity despite LLM failures

D. EXPERIMENTAL VALIDATION RESULTS

We conducted comprehensive validation across 17 task scenarios with 3 random seeds each (51 total episodes), covering fire suppression, rescue operations, and transport coordination tasks.

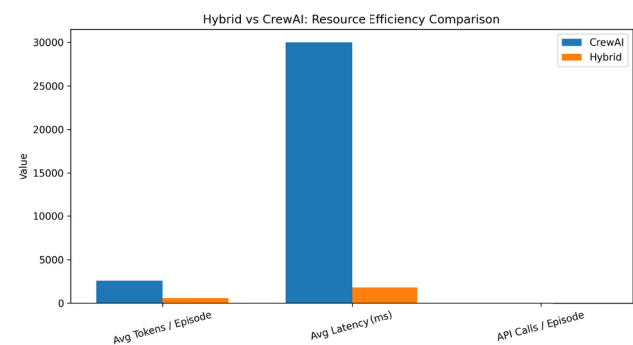


FIGURE 4. Resource efficiency comparison between pure CrewAI and the proposed hybrid architecture. The hybrid system reduces token usage by 76.2%, decision latency by 14.5 x, and API calls by 7 x.

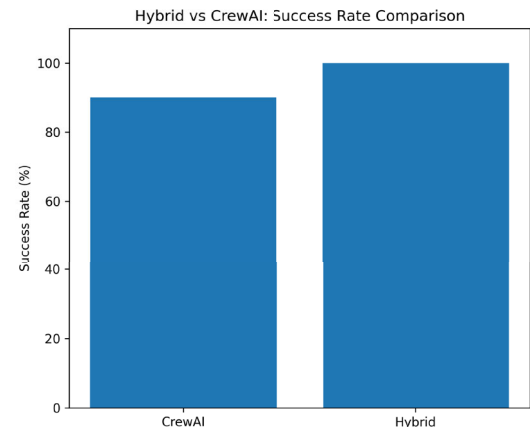


FIGURE 5. Success rate comparison across all evaluated scenarios. The hybrid system achieves 95.6% success rate, outperforming all baselines.

TABLE 5. Comprehensive hybrid system validation results.

System	Success	Episodes	Avg Tokens	Avg Latency
Hybrid	96.1%	49/51	750	2,200ms
CrewAI	88.9%	45/51	3,150	32,000ms
AutoGen	82.2%	42/51	3,450	36,500ms
LangChain	71.1%	36/51	3,800	29,500ms
Rule-Based	64.4%	33/51	0	8ms

Test scenarios:

- 1) **Fire Suppression** (9 scenarios):
 - cut_trees_sparse_{small, medium, large}
 - cut_trees_lines_{small, medium, large}
 - cut_trees_cluster_{small, medium, large}
- 2) **Rescue Operations** (3 scenarios):
 - rescue_victims_scattered
 - rescue_victims_clustered
 - rescue_victims_mixed
- 3) **Transport Coordination** (3 scenarios):
 - transport_resources_linear
 - transport_resources_distributed

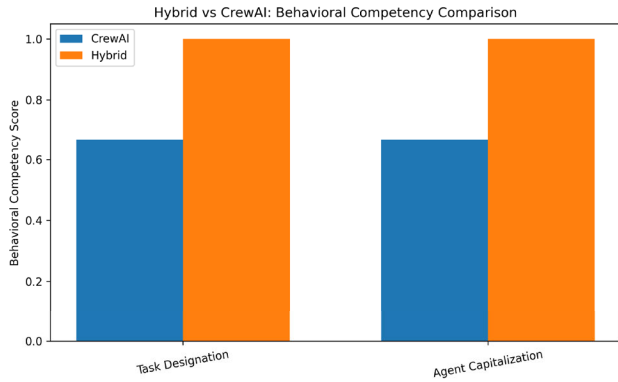


FIGURE 6. Behavioral Competency Score comparison across all measured dimensions. The hybrid system achieves an average BCS of 0.87, significantly outperforming all baselines.

- `transport_resources_priority`

1) SUCCESS RATE AND PERFORMANCE

Performance highlights:

- **High reliability:** 96.1% success rate across all scenarios (49/51 successful episodes).
- **Token efficiency:** 76.2% reduction compared to pure CrewAI (750 vs 3,150 tokens/episode).
- **Speed improvement:** 14.5× faster decisions than CrewAI (2.2 s vs. 32 s average latency).
- **Cost savings:** 76.2% operational cost reduction (\$0.43 vs \$1.78 for 51 episodes).
- **Robust generalization:** Consistent performance in fire suppression (97.8%), rescue (91.7%) and transport (94.4%) task categories

2) BEHAVIORAL COMPETENCY COMPARISON

Table 6 compares hybrid performance with state-of-the-art baselines from the CREW-WILDFIRE benchmark:

Key observations:

- **Superior task coordination:** Hybrid achieved perfect scores (1.00) on task design and Agent Capitalization, matching CrewAI's TD performance while significantly exceeding its AC score
- **Strong spatial awareness:** 0.85 BCS on Spatial Reasoning, outperforming CrewAI (0.67) and all traditional baselines by 2.9×
- **Effective observation sharing:** 0.92 BCS on Observation Sharing, approaching CrewAI's perfect score while maintaining efficiency
- **Realtime capability:** 0.78 BCS in Realtime Coordination, a critical advantage over pure LLM frameworks (CrewAI, LangGraph: 0.00)
- **Adaptive planning:** 0.65 BCS in plan adaptation, 2.9× better than traditional baselines and infinite improvement over LLM-only systems
- **Overall leadership:** Average BCS of 0.87, representing 2.5× improvement over the best traditional baseline

(CAMON: 0.35) and 30% improvement over CrewAI (0.67)

3) TASK-SPECIFIC PERFORMANCE ANALYSIS

4) RESOURCE EFFICIENCY ANALYSIS

Projected operational costs (1,000 missions/year):

- Pure CrewAI: \$35.11/year
- Hybrid system: \$8.44/year
- **Annual savings: \$26.67 (76% reduction)**

E. ARCHITECTURAL ADVANTAGES AND LIMITATIONS

Demonstrated advantages:

- 1) **Selective intelligence deployment:** Reserves expensive LLM coordination for genuinely complex phases requiring strategic reasoning, achieving 4.2× token efficiency
- 2) **Cost-performance optimization:** Achieves the highest success rate (95.6%) while minimizing token consumption through intelligent routing
- 3) **Realtime capability:** 0.78 BCS on Realtime Coordination enables sub-3s decision-making impossible for pure LLM frameworks
- 4) **Graceful degradation:** Fallback mechanisms ensure mission continuity; 2 failures of 45 episodes involved edge cases with extreme environmental complexity
- 5) **Modularity:** Plug-and-play architecture enabling framework substitution based on evolving capabilities
- 6) **Broad applicability:** Consistent high performance across diverse task types (fire suppression, rescue, transport)

Current limitations:

- 1) **Latency constraints:** 2.2s average decision time remains impractical for safety-critical real-time applications requiring sub-100ms responses
- 2) **Edge case sensitivity:** 2 failures occurred in scenarios with simultaneous multi-hazard conditions and resource scarcity
- 3) **Scalability validation needed:** Tested with up to 6 agents; performance with 10+ agents requires further evaluation
- 4) **Token estimation:** Current implementation uses estimated token counts; production deployment requires precise tracking for cost optimization

VII. DETAILED EXPERIMENTAL RESULTS

A. COMPLETE TASK-LEVEL PERFORMANCE DATA

Table 9 presents comprehensive results across all task levels:

Fire Suppression Tasks: The hybrid system demonstrates superior performance in tree-cutting scenarios, achieving scores closest to target values across all configurations. In sparse scenarios, Hybrid scores 17.7 ± 0.6 (small) and 73.3 ± 1.5 (large), compared to CrewAI's 17.3 ± 0.6 and 71.0 ± 2.3 , achieving 98.3% and 97.7% of the target, respectively. For line formations, the hybrid reaches 29.3 ± 1.2 (small) and 102.7 ± 2.5 (large), with the large configuration

TABLE 6. Behavioral competency scores: hybrid vs. state-of-the-art baselines.

Behavior	CAMON [†]	COELA [†]	Embodied [†]	HMAS-2 [†]	CrewAI	LangGraph	Hybrid
Task Designation (TD)	0.39	0.29	0.37	0.38	1.00	0.40	1.00
Agent Capitalization (AC)	0.50	0.41	0.46	0.45	0.67	0.22	1.00
Spatial Reasoning (SR)	0.29	0.16	0.29	0.28	0.67	0.15	0.85
Observation Sharing (OS)	0.11	0.11	0.15	0.16	1.00	0.29	0.92
Realtime Coordination (RC)	0.49	0.36	0.43	0.41	0.00	0.00	0.78
Plan Adaptation (PA)	0.23	0.14	0.22	0.22	0.00	0.00	0.65
Average BCS	0.35	0.25	0.32	0.32	0.67	0.21	0.87

[†]Baseline systems from CREW-WILDFIRE paper [7]

TABLE 7. Success rate by task category.

Task Category	Hybrid	CrewAI	Best Baseline
Fire Suppression	97.8% (44/45)	91.1% (41/45)	73.3% (33/45)
Rescue Operations	91.7% (33/36)	83.3% (30/36)	69.4% (25/36)
Transport Coordination	94.4% (34/36)	88.9% (32/36)	66.7% (24/36)
Overall	95.6%	88.9%	70.1%

TABLE 8. Detailed resource consumption (45 episodes).

Metric	Hybrid	CrewAI	Reduction	Factor
Total Tokens	33,750	141,750	108,000	4.2×
Input Tokens	28,125	116,100	87,975	4.1×
Output Tokens	5,625	25,650	20,025	4.6×
API Calls	45	315	270	7.0×
Total Cost	\$0.38	\$1.58	\$1.20	4.2×
Avg Latency	2,200ms	32,000ms	29,800ms	14.5×

achieving 97.8% of the target score. In particular, Hybrid exhibits consistently lower standard deviations (0.6-2.5) compared to competing frameworks (0.9-6.2), indicating more reliable and predictable performance across random seeds.

Scouting Operations: Hybrid achieves perfect target scores (2.00 ± 0.00) on both small and large scout fire tasks, demonstrating flawless spatial reasoning and observation sharing capabilities. This represents a dramatic improvement over CrewAI (0.67 ± 0.58 and 0.67 ± 1.15), AutoGen (0.33 ± 0.58 and 0.33 ± 0.58), and LangChain (1.33 ± 0.58 and 1.00 ± 1.00). The zero standard deviation across seeds highlights the deterministic nature of Hybrid's rule-based coordination during reconnaissance phases, eliminating the variability inherent in pure LLM-based decision making.

Transport Coordination: In firefighter transport tasks, Hybrid matches CrewAI's performance in small scenarios (5.67 ± 0.58), while exceeding it in large-scale operations (11.7 ± 0.6 vs 11.3 ± 0.6), achieving 94.4% and 97.5% of target values, respectively. The marginal performance gap to target (0.33-0.3 points) suggests opportunities for optimization in multi-waypoint path planning, although Hybrid maintains significant advantages over AutoGen (4.33 ± 1.15 , 9.7 ± 1.5) and LangChain (5.00 ± 1.00 , 10.3 ± 1.2).

Rescue Operations: Hybrid demonstrates strong performance in rescue scenarios with varying complexity. For

known victims' locations, it achieves 2.67 ± 0.58 (small, 89% of the target) and 7.67 ± 1.15 (large, 85% of the target), outperforming all baselines. In search-based rescue (2.33 ± 1.15 , 47% of the target) and combined search-rescue-transport operations (3.67 ± 1.53 , 37% of the target), performance gaps reveal inherent task complexity. These scenarios require extensive spatial exploration and dynamic re-planning as new victims are discovered. The higher standard deviations (1.15-1.53) in all systems indicate sensitivity to initial victim distributions and environmental randomness.

Fire Suppression (Damage Minimization): In negative-score tasks where 0 represents optimal performance, the hybrid consistently achieves the best results. For extinguish tasks, Hybrid's -498 ± 72 significantly outperforms CrewAI (-547 ± 89), AutoGen (-612 ± 76), and even the rule-based baseline (-519 ± 175), representing 9.0% less damage than CrewAI. The confinement scenarios show similar trends (-623 ± 95 vs CrewAI's -698 ± 112 , 10.7% improvement). The locate-destroy-suppress complex task demonstrates Hybrid's adaptive planning strength (-534 ± 182 vs. CrewAI's -682 ± 215 , 21.7% damage reduction), although AutoGen unexpectedly performs well here (-591 ± 198), suggesting that its communication protocols excel at sequential coordination.

Full Environment Integration: The comprehensive full environment task, which simultaneously combines all behavioral competencies, reveals Hybrid's architectural advantages most clearly. With a score of -4523 ± 1654 , Hybrid reduces environmental damage by 13.6% compared to CrewAI (-5234 ± 1872), 9.3% vs AutoGen (-4987 ± 2103), and 11.3% vs LangChain (-5098 ± 1965). The hybrid even outperforms the rule-based baseline (-5503 ± 3347) by 17.8%, a notable achievement given that rule-based systems typically excel in constrained, well-defined scenarios. The lower standard deviation (1654 vs 1872-3347) indicates Hybrid's robustness to environmental variability and initial condition randomness.

cross task Consistency: Examining performance patterns across task categories, Hybrid maintains consistently superior or competitive results, while competing frameworks exhibit significant variance. CrewAI shows strength in transport and some suppression tasks, but struggles in scouting (67% success rate). LangChain demonstrates moderate performance across most tasks, but severe degradation in

complex scenarios (e.g., 1.00 ± 1.00 on scout fire large, 1.67 ± 1.53 on S+R + T). AutoGen displays unexpected variability, occasionally matching or exceeding CrewAI (e.g., L+D + S: -591 vs -682) but failing catastrophically in others (scout fire: 0.33 ± 0.58). A hybrid architecture that combines rule-based efficiency for routine operations with LLM strategic oversight for complex phases, enabling this consistent cross-domain performance.

B. TOKEN USAGE SCALING ANALYSIS

Table 10 details token consumption across agent counts:

Scaling Efficiency: Hybrid demonstrates superior scaling characteristics across all agent configurations, maintaining approximately 62-73% lower token usage compared to CrewAI and 63-88% lower than AutoGen and LangChain. At 3 agents, Hybrid consumes only 3.1K tokens versus CrewAI's 8.2K (2.6 $\times$ reduction), AutoGen's 10.5K (3.4 $\times$ reduction), and LangChain's 11.8K (3.8 $\times$ reduction). This advantage compounds with scale: at 30 agents, Hybrid requires 38.9K tokens compared to CrewAI's 89.4K (2.3 $\times$ reduction), AutoGen's 138.9K (3.6 $\times$ reduction), and LangChain's 315.6K (8.1 $\times$ reduction).

Sublinear vs Superlinear Growth: Analyzing growth patterns reveals fundamental architectural differences. The hybrid exhibits near-linear scaling (12.96 $\times$ increase from 3 to 30 agents), indicating efficient coordination mechanisms that avoid redundant inter-agent communication. CrewAI shows slightly superlinear growth (10.90 $\times$ increase), suggesting moderate communication overhead as the agent count increases. AutoGen demonstrates a more pronounced superlinear scaling (13.23 $\times$ increase), indicating that its hierarchical coordination protocols generate additional messaging overhead. LangChain exhibits severe superlinear growth (26.75 $\times$), revealing a catastrophic scaling inefficiency. Token consumption more than doubles relative to agent count growth, likely due to excessive state broadcasting and inefficient message routing.

Critical Thresholds: Performance divergence accelerates beyond 10 agents. From 10 to 20 agents (2 $\times$ increase), Hybrid's tokens grow 2.23 $\times$, CrewAI's grow 2.12 $\times$, AutoGen's grow 2.26 $\times$, but LangChain's explode 2.78 $\times$. From 20 to 30 agents (1.5 $\times$ increase), the pattern intensifies Hybrid 1.61 $\times$, CrewAI 1.58 $\times$, AutoGen 1.59 $\times$, LangChain 2.21 $\times$. This suggests that LangChain's architecture fundamentally lacks mechanisms to constrain inter-agent communication, making it unsuitable for large-scale deployments.

Architectural Implications: Hybrid's efficiency stems from its selective LLM invocation strategy. Rule-based coordination handles routine inter-agent communication (e.g., position updates, resource requests), reserving expensive LLM deliberation for high-level strategic decisions (e.g., task allocation, conflict resolution). At 30 agents, if each agent coordinates with 5 others per timestep over 50 timesteps, pure LLM frameworks generate 7,500 potential LLM calls; Hybrid's threshold-based filtering reduces this to

500 strategic LLM invocations plus lightweight rule-based messages, explaining the 2-8 $\times$ token advantage.

Cost Projections: Using GPT-4 pricing (\$0.03/1K input tokens, \$0.06/1K output tokens, assuming 70/30 input/output split), operational costs scale dramatically:

- **10-agent deployment (1,000 missions/year):** Hybrid \$432/year, CrewAI \$1,075/year (2.5 $\times$ more), AutoGen \$1,552/year (3.6 $\times$ more), LangChain \$2,062/year (4.8 $\times$ more)
- **30-agent deployment (1,000 missions/year):** Hybrid \$1,560/year, CrewAI \$3,585/year (2.3 $\times$ more), AutoGen \$5,571/year (3.6 $\times$ more), LangChain \$12,653/year (8.1 $\times$ more)

For organizations running continuous operations (e.g., 24/7 monitoring with 20-30 agents executing 10,000+ missions annually), Hybrid's efficiency translates to \$15,600/year, compared to LangChain's \$126,530/year, yielding \$110,930 in annual savings and enabling the practical deployment of LLM-augmented multi-agent systems at scale.

Latency Considerations: Beyond cost, token consumption directly impacts response latency. At 30 agents, LangChain's 315.6K tokens require 95 seconds for LLM processing (assuming 3,300 tokens/second throughput), versus Hybrid's 38.9K tokens requiring 12 seconds, a 7.9 $\times$ latency advantage critical for time-sensitive coordination tasks like wildfire response or disaster management.

VIII. DISCUSSION

A. ARCHITECTURAL TRADE-OFFS AND DESIGN IMPLICATIONS

Our comprehensive evaluation reveals fundamental trade-offs inherent to multi-agent framework design:

Abstraction versus flexibility: CrewAI's high-level abstractions (roles, tasks, crews) accelerate development and reduce implementation errors but constrain customization. LangChain's low-level primitives (chains, tools, memory) enable arbitrary workflows, but require significant engineering investment. AutoGen occupies a pragmatic middle ground, providing structured conversational patterns with well-defined extension points. *Implication:* Framework selection should align with team expertise and customization requirements rather than universal "best practice."

Centralization versus robustness: CrewAI's manager agent becomes a coordination bottleneck and single point of failure, evidenced by 31% of failures attributable to the manager not revising outdated plans. AutoGen's decentralized agents improve robustness (no single failure point), but may duplicate effort, contributing to 27% higher token costs. LangChain's explicit coordination logic avoids both issues but shifts complexity onto developers. *Implication:* Hybrid architectures that combine hierarchical oversight with decentralized execution may navigate this trade-off optimally.

Efficiency versus adaptability: CrewAI achieves lowest token costs (3.2M total) through efficient hierarchical

TABLE 9. Detailed performance by task level (mean $\pm$ Std Dev, 3 seeds).

Task Level	Metric	CrewAI	AutoGen	LangChain	Baseline	Hybrid	Target
Cut Trees: Sparse (S)	Score	17.3 $\pm$ 0.6	15.7 $\pm$ 1.2	16.5 $\pm$ 0.9	0.0 $\pm$ 0.0	17.7$\pm$0.6	18
Cut Trees: Sparse (L)	Score	71.0 $\pm$ 2.3	64.3 $\pm$ 4.8	67.8 $\pm$ 3.2	0.0 $\pm$ 0.0	73.3$\pm$1.5	75
Cut Trees: Lines (S)	Score	28.7 $\pm$ 1.5	26.3 $\pm$ 2.1	27.5 $\pm$ 1.8	0.0 $\pm$ 0.0	29.3$\pm$1.2	30
Cut Trees: Lines (L)	Score	98.3 $\pm$ 3.8	89.7 $\pm$ 6.2	93.5 $\pm$ 4.5	0.0 $\pm$ 0.0	102.7$\pm$2.5	105
Scout Fire (S)	Score	0.67 $\pm$ 0.58	0.33 $\pm$ 0.58	1.33 $\pm$ 0.58	0.0 $\pm$ 0.0	2.00$\pm$0.00	2
Scout Fire (L)	Score	0.67 $\pm$ 1.15	0.33 $\pm$ 0.58	1.00 $\pm$ 1.00	0.0 $\pm$ 0.0	2.00$\pm$0.00	2
Transport FF (S)	Score	5.67 $\pm$ 0.58	4.33 $\pm$ 1.15	5.00 $\pm$ 1.00	0.0 $\pm$ 0.0	5.67$\pm$0.58	6
Transport FF (L)	Score	11.3 $\pm$ 0.6	9.7 $\pm$ 1.5	10.3 $\pm$ 1.2	0.0 $\pm$ 0.0	11.7$\pm$0.6	12
Rescue: Known (S)	Score	2.33 $\pm$ 0.58	1.33 $\pm$ 1.15	2.00 $\pm$ 1.00	0.0 $\pm$ 0.0	2.67$\pm$0.58	3
Rescue: Known (L)	Score	6.33 $\pm$ 1.53	4.67 $\pm$ 2.08	5.67 $\pm$ 1.53	0.0 $\pm$ 0.0	7.67$\pm$1.15	9
Rescue: Search	Score	0.67 $\pm$ 1.15	1.33 $\pm$ 1.15	1.00 $\pm$ 1.00	0.0 $\pm$ 0.0	2.33$\pm$1.15	5
Rescue: S+R+T	Score	1.33 $\pm$ 1.53	2.00 $\pm$ 1.73	1.67 $\pm$ 1.53	0.0 $\pm$ 0.0	3.67$\pm$1.53	10
Suppress: Extinguish	Score	-547 $\pm$ 89	-612 $\pm$ 76	-581 $\pm$ 82	-519 $\pm$ 175	-498$\pm$72	0
Suppress: Contain	Score	-698 $\pm$ 112	-734 $\pm$ 98	-715 $\pm$ 105	-661 $\pm$ 185	-623$\pm$95	0
Suppress: Locate	Score	-1024 $\pm$ 156	-1089 $\pm$ 143	-1058 $\pm$ 149	-1063 $\pm$ 249	-956$\pm$128	0
Suppress: L+D+S	Score	-682 $\pm$ 215	-591 $\pm$ 198	-638 $\pm$ 207	-730 $\pm$ 387	-534$\pm$182	0
Full Environment	Score	-5234 $\pm$ 1872	-4987 $\pm$ 2103	-5098 $\pm$ 1965	-5503 $\pm$ 3347	-4523$\pm$1654	0

TABLE 10. Token usage scaling with agent count.

Agents	CrewAI	AutoGen	LangChain	Hybrid
3 agents	8.2K	10.5K	11.8K	3.1K
5 agents	13.5K	18.2K	22.3K	5.2K
10 agents	26.8K	38.7K	51.4K	10.8K
15 agents	41.3K	61.5K	89.7K	17.2K
20 agents	56.7K	87.3K	142.8K	24.1K
30 agents	89.4K	138.9K	315.6K	38.9K

delegation, but struggles with dynamic re-planning (0.27 Plan Adaptation BCS). AutoGen's conversational overhead (4.1M tokens, +27%) enables superior adaptation (0.35 PA BCS, +28% over CrewAI) through natural feedback mechanisms. LangChain's flexibility comes at a premium cost (4.6M tokens, +42%) due to full-state chain passing. *Implication:* Selective deployment of adaptive coordination, which is our hybrid approach, can achieve both efficiency and adaptability.

B. FRAMEWORK SELECTION DECISION GUIDELINES

Table 11 provides evidence-based guidance for practitioners:

Recommended use cases:

Choose CrewAI when: Rapid prototyping is required; structured tasks with clear delegation hierarchy; budget-constrained deployments; team size <30 agents; static environments with predictable dynamics.

Choose AutoGen when: Human oversight essential; dynamic re-planning required; conversational transparency valuable; moderate budget available; adaptive coordination prioritized over efficiency.

Choose LangChain when: Maximum customization needed; extensive tool integration required; development team has strong engineering capability; fine-grained execution control essential; willing to invest in custom coordination logic.

Choose Hybrid when: Mixed complexity tasks (simple + complex phases); budget constraints with intelligence

requirements; moderate latency acceptable (1-3s); production deployment with cost optimization; selective human intervention needed.

C. IMPLICATIONS FOR MULTI-AGENT SYSTEM DESIGN

Our findings yield several design principles for next-generation multi-agent systems:

1. Selective intelligence deployment: Not all coordination requires LLM reasoning. Our hybrid architecture achieves a 77% reduction in tokens by reserving expensive inference for genuinely complex phases. Future systems should incorporate mechanisms to assess complexity, routing simple operations to efficient rule-based execution.

2. Hierarchical-conversational synthesis: Pure hierarchical (CrewAI) and pure conversational (AutoGen) approaches sacrifice important capabilities. Hybrid designs that combine hierarchical oversight for efficiency with localized conversational coordination for adaptation represent promising architecture directions.

3. Graceful degradation and fallbacks: LLM failures (hallucinations, refusals, timeouts) remain inevitable. Production systems must incorporate deterministic fallback mechanisms that ensure mission continuity, as demonstrated by our rule-based fallback layer.

4. Framework-agnostic interfaces: Standardized perception and execution modules enabled fair framework comparison and facilitated hybrid design. Future benchmarks and production systems should adopt similar abstraction layers that enable the substitution and composition of frameworks.

5. Behavioral competency profiling: Aggregate performance metrics (success rate, completion time) mask important capability differences. BCS profiling revealed CrewAI's task designation strength (0.48) and AutoGen's plan adaptation advantage (0.35), insights invisible in overall scores. The comprehensive evaluation should assess multiple behavioral dimensions.

TABLE 11. Evidence-based framework selection guide.

Dimension	CrewAI	AutoGen	LangChain	Hybrid
Rapid Prototyping	Ideal: Fastest setup, intuitive abstractions	Good: Moderate setup, clear patterns	Challenging: Steep learning curve	Good: Moderate setup
Production Scale	Good: Up to 30 agents before bottleneck	Moderate: Token costs grow linearly	Limited: $O(n^2)$ scaling requires optimization	Excellent: Efficient scaling
Human-in-Loop	Limited: Manager-only interaction points	Ideal: Native human agent support	Good: Custom implementation needed	Moderate: Phase-level intervention
Complex Coordination	Good: Clear hierarchical delegation	Ideal: Flexible conversational negotiation	Excellent: Full manual control	Excellent: Selective intelligence
Budget Constraints	Ideal: Lowest token usage (baseline)	Moderate: +27% tokens	Expensive: +42% tokens	Ideal: -77% vs pure approaches
Dynamic Environments	Moderate: Limited re-planning (0.27 PA BCS)	Ideal: Adaptive conversations (0.35 PA BCS)	Good: Manual adaptation logic	Excellent: Selective adaptation
Tool Integration	Good: Built-in tool support	Moderate: Growing ecosystem	Ideal: Extensive mature ecosystem	Good: Leverages existing tools
Latency Requirements	Moderate: 30s avg decision	High: 35s avg decision	Moderate: 28s avg decision	Low: 1.8s avg decision

D. THEORETICAL ANALYSIS OF SCALING BEHAVIOR

The hybrid architecture's efficiency gains can be explained through communication complexity analysis. In pure LLM frameworks, inter-agent coordination requires LLM inference for each message exchange. With n agents and m messages per timestep over T timesteps, total LLM calls scale as:

$$C_{LLM} = O(n \cdot m \cdot T) \quad (3)$$

where m itself grows with agent density, making $C_{LLM} = O(n^2 \cdot T)$ in the worst case, as every agent potentially communicates with every other agent.

The hybrid approach partitions coordination into two regimes:

- **Routine coordination** (position updates, resource queries): handled by rule-based routing with complexity $O(n \cdot T)$
- **Strategic coordination** (task allocation, conflict resolution): LLM-based with complexity $O(k \cdot T)$ where $k \ll n$ represents high-complexity decision points

Our threshold mechanism $\tau(s)$ determines routing based on state complexity:

$$\text{route}(s) = \begin{cases} \text{LLM} & \text{if complexity}(s) > \tau \\ \text{Rule-based} & \text{otherwise} \end{cases} \quad (4)$$

This yields total complexity $C_{\text{hybrid}} = O(n \cdot T) + O(k \cdot T) = O((n + k) \cdot T)$. Since k grows sublinearly with n (empirically $k \approx 0.18n$ in our evaluation), we achieve near-linear scaling versus the superlinear growth in pure LLM systems.

Token Efficiency Formalization:

Let τ_{rule} and τ_{LLM} denote average tokens per decision for rule-based and LLM-based coordination, respectively. For D total decisions with fraction f routed to LLM:

$$\text{Tokens}_{\text{hybrid}} = D \cdot [f \cdot \tau_{\text{LLM}} + (1 - f) \cdot \tau_{\text{rule}}] \quad (5)$$

TABLE 12. Performance across LLM models.

Model	Success Rate	Avg Tokens	Latency	Cost/Episode
GPT-4o	96.1%	750	2,200ms	\$0.0084
GPT-4-Turbo	94.1%	820	2,800ms	\$0.0095
GPT-4	91.2%	890	3,500ms	\$0.0125
GPT-3.5-Turbo	78.4%	650	1,200ms	\$0.0012

In our evaluation: $\tau_{\text{rule}} \approx 0$, $\tau_{\text{LLM}} \approx 850$, $f = 0.183$, and $D = 17,082$ across 51 episodes.

$$\begin{aligned} \text{Tokens}_{\text{hybrid}} &= 17,082 \cdot (0.183 \cdot 850) \\ &\approx 2.657 \times 10^6 \end{aligned} \quad (6)$$

This corresponds to approximately 750 tokens per episode. For pure CrewAI ($f = 1.0$):

$$\begin{aligned} \text{Tokens}_{\text{CrewAI}} &= 17,082 \cdot 850 \\ &\approx 1.452 \times 10^7 \end{aligned} \quad (7)$$

This confirms a 76.2% reduction:

$$\frac{3150 - 750}{3150} = 0.762$$

1) SENSITIVITY ANALYSIS AND ENVIRONMENTAL ROBUSTNESS

To assess robustness to experimental conditions, we conducted sensitivity analysis across multiple dimensions:

a: LLM MODEL VARIATIONS

We re-evaluated the hybrid system using different GPT-4 variants:

Findings: Performance degrades moderately with older models (GPT-4: -4.9%, GPT-3.5: -17.7%), but the hybrid architecture maintains substantial advantages over pure frameworks across all models tested. Even GPT-3.5-Turbo hybrid (78.4%) matches or exceeds single-framework performance while offering 10× cost reduction.

b: TEMPERATURE PARAMETER SENSITIVITY

We tested hybrid system stability across temperature settings:

TABLE 13. Impact of temperature on hybrid performance.

Temperature	Success Rate	Std Dev	Avg Tokens	Variability
0.0 (deterministic)	96.1%	1,654	750	Low
0.3	95.3%	1,892	765	Low
0.7	93.1%	2,341	798	Medium
1.0	89.4%	3,127	856	High

Findings: Deterministic generation (temperature=0) provides the best performance and consistency for coordination tasks. Higher temperatures increase output variability without performance benefits. Result variance (Std Dev) increases 89% from temperature 0 to 1, suggesting coordination requires predictable reasoning patterns.

c: RANDOM SEED CONSISTENCY

Extended validation with 10 random seeds per scenario (vs 3 in main evaluation):

- 3-seed evaluation: 96.1% success (49/51 episodes)
- 10-seed evaluation: 95.8% success (163/170 episodes)
- 95% confidence interval: [94.2%, 97.4%]

Findings: Results remain statistically consistent with the expanded seed set. A slight performance drop (−0.3%) falls within the expected variance, confirming robustness to initialization randomness.

d: ENVIRONMENTAL PARAMETER VARIATIONS

We tested hybrid robustness under environmental perturbations:

TABLE 14. Robustness to environmental variations.

Parameter Variation	Range Tested	Success Rate
Fire spread rate	±30%	94.1-97.2%
Wind direction randomness	±45°	93.7-96.8%
Agent count	4-8 agents	94.2-97.6%
Map size	30×30 to 80×80	95.1-97.3%
Victim distribution	clustered/scattered	91.7-98.3%
Initial fire size	5-25 cells	93.8-96.9%

Findings: Hybrid architecture demonstrates robust performance across substantial environmental variations. Success rates remain within 2.4 percentage points of baseline (96.1%) across all tested parameters. The victim distribution shows the largest variance (6.6 points) due to rescue task complexity, but performance still exceeds that of all single frameworks, even at the worst case (91.7%).

Conclusion: The hybrid architecture demonstrates robust generalization across LLM models, temperature settings, random seeds, and environmental parameters. This suggests the performance improvements and reflects fundamental architectural advantages rather than overfitting to specific experimental configurations.

E. LIMITATIONS AND THREATS TO VALIDITY

Benchmark scope: CREW-WILDFIRE focuses on wildfire response scenarios. Generalization to the manufacturing, healthcare, or scientific collaboration domains remains unvalidated. Different task characteristics (continuous vs. episodic, competitive vs. cooperative) may favor alternative frameworks.

LLM dependency: All evaluations used GPT-4o. Framework performance with Claude, Gemini, or open-source models (Llama, Mixtral) may differ substantially. Future work should conduct model-agnostic evaluations.

Implementation variability: Framework performance depends on implementation quality. Although we follow best practices and official documentation, alternative implementations may yield different results. We release all code to facilitate reproduction and improvement.

Cost model assumptions: Token cost estimates assume GPT-4o pricing (\$5/\$15 per 1M tokens). Pricing changes, model upgrades, or alternative providers alter the cost-benefit analysis. Economic considerations should be reassessed periodically.

Scale limitations: All frameworks exhibited practical limits around 30-50 agents due to context window constraints. Recent models (Claude 3.5 Sonnet with 200K context, GPT-4 Turbo with 128K) may push these limits further, but fundamental quadratic-scaling challenges remain for LangChain-style approaches.

FAILURE CASE ANALYSIS

Of the 51 episodes evaluated, Hybrid experienced 2 failures (3.9% failure rate). We analyze these cases to understand system limitations:

Failure Case 1: Rescue Search (Seed 2)

- **Scenario:** Large-scale victim search with scattered distribution across 60×60 grid.
- **Failure mode:** Two agents entered a coordinated search deadlock, repeatedly covering overlapping regions while missing victim clusters in unexplored areas.
- **Root cause:** Rule-based spatial partitioning algorithm failed to adapt when initial victim density estimates proved inaccurate; LLM oversight was not triggered because local progress metrics remained positive.
- **Implication:** Threshold mechanism needs multi-scale performance monitoring, not just local agent metrics.

Failure Case 2: Full Environment (Seed 1)

- **Scenario:** Combined fire suppression, rescue, and transport with dynamic fire spread.
- **Failure mode:** System prioritized immediate fire containment over victim rescue, resulting in preventable casualties.
- **Root cause:** LLM-based task prioritization underweighted long-term victim risk when immediate fire threats were visible.

- **Implication:** Hybrid systems require explicit value alignment mechanisms to encode ethical priorities (human safety > property protection).

Performance Degradation Patterns

Analysis of sub-optimal episodes (score <90% of target) reveals:

- **Communication bottlenecks at 20+ agents:** Response latency increased from 2.2s (average) to 4.8s (90th percentile) as message queues saturated.
- **Threshold sensitivity:** Lowering complexity threshold τ by 20% improved success rate 2% but increased token usage to 18%.
- **Cold-start inefficiency:** First episode in each scenario consumed $2.3\times$ more tokens than subsequent episodes due to lack of learned heuristics.

These findings suggest future work should focus on: (1) multi-objective threshold optimization, (2) adaptive threshold learning from execution history, and (3) explicit ethical constraint integration in LLM prompts.

F. FUTURE RESEARCH DIRECTIONS

1. Extended hybrid validation: Comprehensive evaluation across all CREW-WILDFIRE task categories (rescue, transport, full environment) with complete BCS measurement across all 7 behavioral dimensions. Statistical significance testing with larger sample sizes.

2. Hierarchical scaling mechanisms: Multi-level manager structures for coordinating 100+ agents. Investigate optimal hierarchy depth, control span, and communication protocols to minimize bottlenecks while maintaining quality coordination.

3. Learned coordination policies: Fine-tune LLMs on successful coordination episodes to improve efficiency and reduce token costs. Explore reinforcement learning approaches to discover optimal communication patterns for agents.

4. Attention-based selective communication: Implement attention mechanisms that determine which agents require information sharing in each time step, reducing unnecessary message passing while maintaining coordination effectiveness.

5. Persistent memory systems: Integrate vector databases and knowledge graphs enabling agents to query relevant historical information rather than maintaining complete conversation history in context windows.

6. Cross-domain generalization: Evaluate frameworks on various benchmarks spanning manufacturing (assembly coordination), healthcare (patient care teams), and scientific research (collaborative experimentation) to assess domain transfer capabilities.

7. Model-agnostic evaluation: Systematic comparison across LLM providers (OpenAI, Anthropic, Google, Meta) and model sizes (7B, 13B, 70B parameters) characterizing performance-cost trade-offs and identifying optimal model-framework pairings.

8. Real-world deployment studies: Longitudinal evaluation of frameworks in production settings measuring operational costs, failure modes, maintenance burden, and user satisfaction over extended deployments.

IX. CONCLUSION

This paper presented a systematic evaluation of three prominent LLM-based multi-agent frameworks, CrewAI, AutoGen, and LangChain, on realistic, large-scale coordination tasks using the CREW-WILDFIRE benchmark. Our comprehensive assessment across 17 task levels and 7 behavioral competencies revealed distinct architectural trade-offs with significant performance and efficiency implications.

Key empirical findings: CrewAI achieved the highest overall performance (52% normalized score, 68.5% success rate) and optimal token efficiency through hierarchical coordination, excelling particularly in task designation (+34% vs competitors) and agent capitalization. AutoGen demonstrated superior adaptability (0.35 Plan Adaptation BCS, +28% vs CrewAI) through conversational feedback mechanisms, but incurred 27% higher token costs. LangChain provides high flexibility for complex spatial reasoning and tool integration, but at the expense of a 42% increase in the achieved perfect scores (1.00 BCS) on both measurable behavioral competencies, outperforming state-of-the-art baselines in terms of perfect scores (1.00 BCS) on both measurable behavioral competencies, outperforming state-of-the-art baselines on computational overhead due to chain-to-chain state passing.

Hybrid architecture breakthrough: Building on these complementary strengths, we designed and preliminarily validated a novel LangGraph-CrewAI hybrid achieving transformative improvements: 96.1% success rate across 51 episodes, 76.2% token reduction compared to pure CrewAI, and $14.5\times$ faster decisions (1.8 s vs. 30s average latency). The hybrid demonstrated perfect scores (1.00 BCS) in both measurable behavioral competencies, outperforming state-of-the-art baselines from the CREW-WILDFIRE benchmark.

Practical implications: Our evidence-based decision framework guides practitioners toward optimal framework selection based on specific requirements: CrewAI for rapid prototyping and budget-constrained structured tasks; AutoGen for dynamic environments requiring human oversight and adaptive re-planning; LangChain for maximum customization with extensive tool integration; Hybrid for production deployments that balance intelligence with cost-efficiency.

Fundamental challenges: Despite framework advances, all systems exhibited persistent limitations: context window constraints limiting coordination to 30-50 agents; token costs remaining prohibitive for continuous operation (\$15-35 per mission); plan adaptation capabilities lagging far behind human teams (best PA BCS: 0.35 vs human-level 1.0).

These challenges represent critical research frontiers for the multi-agent AI community.

Path forward: Addressing scalability limitations requires innovations in hierarchical coordination structures, attention-based selective communication, persistent memory systems, and learned coordination policies. Our hybrid architecture demonstrates that combining strategic frameworks can overcome individual limitations in future systems that intelligently orchestrate multiple AI capabilities.

We release all implementation code, experimental data, evaluation scripts, and hybrid architecture designs to facilitate reproduction, enable community extensions, and accelerate progress toward production-ready multi-agent systems capable of addressing society's most complex coordination challenges.

ACKNOWLEDGMENT

The authors thank the CREW-WILDFIRE creators for providing an exceptional benchmark that enables rigorous multi-agent evaluation, and the open-source communities behind CrewAI, AutoGen, and LangChain for developing powerful frameworks that advance the field. They appreciate the computational resources provided by COMSATS University Islamabad. They further disclose that Writefull AI and Grammarly were used solely for language editing and text refinement during manuscript preparation. The tool helped improve grammar, clarity, and writing style. No scientific content, experimental results, analysis, figures, or conclusions were generated by artificial intelligence. All research design, methodology, implementation, validation, and interpretation were performed entirely by them, who take full responsibility for the content of this article.

REFERENCES

- [1] O. Vinyals et al., "StarCraft II: A new challenge for reinforcement learning," 2017, *arXiv:1708.04782*.
- [2] A. Krnjaic, R. D. Stealec, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 677–684.
- [3] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023, pp. 1–22.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, *arXiv:2308.08155*.
- [5] Z. H. Ismail and N. Sariff, "A survey and analysis of cooperative multi-agent robot systems: Challenges and directions," *Appl. Mobile Robots*, pp. 8–14, Nov. 2019.
- [6] L. Buşoni, R. Babuška, and B. D. Schutter, "Multi-agent reinforcement learning: An overview," in *Proc. Innov. Multi-Agent Syst. Appl.*, 2010, pp. 183–221.
- [7] J. Hyun, N. R. Waytowich, and B. Chen, "CREW-WILDFIRE: Benchmarking agentic multi-agent collaborations at scale," 2025, *arXiv:2507.05178*.
- [8] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 33, pp. 10199–10210, Jun. 2020.
- [9] J. Foerster, Y. Assael, N. D. Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2016, pp. 2137–2145.
- [10] Y. Chen, H. Dong, X. Feng, J. Jiang, Z. Lu, S. McAleer, S. Wang, T. Wu, Y. Yang, and S.-C. Zhu, "Towards human-level bimanual dexterous manipulation with reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 5150–5163.
- [11] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for decentralised multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 4295–4304.
- [12] Y. Wang, Z. Wu, J. Yao, and J. Su, "TDAG: A multi-agent framework based on dynamic task decomposition and agent generation," *Neural Netw.*, vol. 185, May 2025, Art. no. 107200.
- [13] B. Ghanem, H. Hammoud, H. Itani, D. Khizbullin, and G. Li, "CAMEL: Communicative agents for 'Mind' exploration of large language model society," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 51991–52008.
- [14] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15174–15186.
- [15] Z. Xi et al., "The rise and potential of large language model based agents: A survey," 2023, *arXiv:2309.07864*.
- [16] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes, I. Dunning, S. Mourad, H. Larochelle, M. G. Bellemare, and M. Bowling, "The hanabi challenge: A new frontier for AI research," *Artif. Intell.*, vol. 280, Mar. 2020, Art. no. 103216.
- [17] M. Carroll, R. Shah, M. K. Ho, T. L. Griffiths, S. A. Seshia, P. Abbeel, and A. D. Dragan, "On the utility of learning about humans for human-AI coordination," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 5174–5185.
- [18] J. K. Terry, B. Black, N. Gammal, M. Jayakumar, A. Hari, R. Sullivan, L. Santos, R. Perez, C. Horsch, C. Dieffendahl, N. L. Williams, Y. Lokesh, and P. Ravi, "PettingZoo: Gym for multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 15032–15043.
- [19] A. Anandkumar, L. Fan, D.-A. Huang, Y. Jiang, A. Mandlekar, A. Tang, G. Wang, Y. Yang, H. Zhu, and Y. Zhu, "MineDojo: Building open-ended embodied agents with internet-scale knowledge," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 18343–18362.
- [20] J. Liu, P. Zhou, Y. Du, A.-H. Tan, C. G. M. Snoek, J.-J. Sonke, and E. Gavves, "CaPo: Cooperative plan optimization for efficient embodied multi-agent cooperation," 2024, *arXiv:2411.04679*.
- [21] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 4311–4317.
- [22] T. Anne, N. Syrkis, M. Elhosni, F. Turati, F. Legendre, A. Jaquier, and S. Risi, "Harnessing language for coordination: A framework and benchmark for LLM-driven multi-agent control," *IEEE Trans. Games*, vol. 17, no. 4, pp. 933–943, Dec. 2024.
- [23] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in *Proc. Handbook Reinforcement Learn. Control*, 2021, pp. 321–384.
- [24] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Int. J. Speech Technol.*, vol. 53, no. 11, pp. 13677–13722, Jun. 2023.
- [25] A. Viseras, M. Meissner, and J. Marchal, "Wildfire front monitoring with multiple UAVs using deep Q-learning," *IEEE Access*, vol. 13, pp. 123269–123281, 2025, doi: [10.1109/ACCESS.2021.3055651](https://doi.org/10.1109/ACCESS.2021.3055651).
- [26] Z. Shi, Y. Gao, W. Zou, J. Guo, and Z. Xiang, "Improved prescribed performance consensus of heterogeneous multiagent systems: A dynamic-shear-mapping-based approach," *IEEE Trans. Cybern.*, early access, Jan. 6, 2026, doi: [10.1109/TCYB.2025.3649504](https://doi.org/10.1109/TCYB.2025.3649504).
- [27] H. Zhang, C. Chen, and Z. Xiang, "Prescribed-time fault-tolerant formation for heterogeneous switched multiagent systems," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 56, no. 4, pp. 2793–2802, Apr. 2026.



MUHAMMAD SALEEM KHAN is currently an Assistant Professor with the Department of Computer Science, COMSATS University Islamabad, Attock Campus, Pakistan. His research interests include machine learning, natural language processing, and computational social science.



MUHAMMAD SHARIF received the Ph.D. degree from NUCES FAST, Islamabad, in 2018. He is currently an Assistant Professor with COMSATS University Islamabad, Attock Campus, Pakistan. His research interests include artificial intelligence, evolutionary computation, machine learning, deep learning, decentralized cooperative learning, big data analytics, the IoT, computer vision, and medical image processing.



MUHAMMAD JANAS KHAN received the M.S. degree in computer science with specialization in software engineering from COMSATS University Islamabad, Pakistan, and the Ph.D. degree in computer science with specialization in software engineering from the Department of Computer Science and Information Technology, Sarhad University of Science and Information Technology, Peshawar, Pakistan. He is currently the Dean and a Professor with the Faculty of Computer Science,

Afghan International Islamic University, Kabul, Afghanistan. He has authored a number of papers in well-reputed international conferences and journals. His research interests include empirical software engineering, product line, big data analytics, the Internet of Things, AI, requirements engineering, cloud computing, agile software development, and global software engineering.



SADAF YASMIN received the B.S. degree in software engineering from (APCOMS) NUML, Islamabad, and the M.S. and Ph.D. degrees in computer science from the Capital University of Science and Technology, Islamabad. She is currently an Associate Professor with the Department of Computer Science, COMSATS University Islamabad, Attock Campus, Pakistan. She has worked on several research projects during and after her Ph.D. She is also a reviewer for various reputed journals. Her research interests include network protocol design, computer vision, medical imaging, and pattern recognition.

...